Functional Dependencies in the TaxiManagementSystem

Tirth Kansagra-202303055

Rom Tala – 202303051

Virat Srimali – 202303061

1. Login Table

- **Functional Dependencies:**
  - LoginID→Username, PasswordHash, Role
- **Normal Form Analysis:**
  - 1NF: Satisfied (all attributes are atomic).
  - 2NF: Satisfied (no partial dependencies; only one primary key).
  - 3NF: Satisfied (no transitive dependencies).
  - BCNF: Satisfied (LoginID is a super key.all 3 BCNF FD's are possible).
- **Conclusion: The Login Table is in BCNF only.**

---

2. Driver Table

- **Functional Dependencies:**
  - DriverID→Name, PhoneNo, Email, AvlStatus, LoginID

    From these we can make various 3NF and BCNF FD's.
  - DriverID->LoginID(3NF)(1 FD is possible)
  - DriverID-> Name, PhoneNo, Email, AvlStatus. **(BCNF)(4 FD's are possibles)**
- **Normal Form Analysis:**
  - 1NF: Satisfied.
  - 2NF: Satisfied (no partial dependencies; only one primary key).
  - 3NF: Satisfied (LoginID is a candidate key, but it does not cause transitive dependencies and it is prime attribute).
  - BCNF: Satisfied (DriverID is a super key).
- **Conclusion: The Driver Table is in BCNF and 3NF.**

---

3. Customer Table

- **Functional Dependencies:**
    - CustomerID→Name, PhoneNo, Email, LoginID

      From these we can make various 3NF and BCNF FD's.
    - CustomerID ->LoginID**(3NF)(1 FD)**
    - CustomerID -> Name, PhoneNo, Email. **(BCNF)(3 FD are possibles.)**
- **Normal Form Analysis:**
    - 1NF: Satisfied.
    - 2NF: Satisfied (no partial dependencies; only one primary key).
    - 3NF: Satisfied (LoginID is a candidate key, but it does not cause transitive dependencies and it is prime attribute).
    - BCNF: Satisfied (CustomerID is a super key).
- **Conclusion: The Customer Table is in BCNF and 3NF.**

---

4. Shift Table (Composite Key: DriverID, ShiftStartTime)

- **FDs:**
    - (DriverID, ShiftStartTime) → Hours
- **Analysis:**
    - 1NF: All attributes (DriverID, ShiftStartTime, Hours) are atomic.
    - 2NF: Since the primary key is a composite of DriverID and ShiftStartTime and Hours depends on the full composite key and not a part of it, so there are no partial dependencies.
    - **3NF: Not Satisfied(as hours is not prime attribute.)**
    - **BCNF: Satisfied (DriverId, ShiftStartTime is a super key).**
- **<u>Conclusion:- The Shift table is only BCNF but not 3NF.</u>**

---

5. RideBooking Table:

- **FDs:**
    - RideID → Fare, DriverID, CustomerID, Status, CurrentLocation, Destination

      From these we can make various 3NF and BCNF FD's.
    - RideID -> CustomerID,DriverID **(3NF) (2 FD's are possible)**
    - RideID -> Name, PhoneNo, Email. **(BCNF)(3 FD's are possible)**

- **Analysis:**
    - 1NF: All attributes (RideID, Fare, DriverID, CustomerID, Status, CurrentLocation, Destination) are atomic.
    - 2NF: RideID is the primary key, and all other attributes depend fully on it, so no partial dependencies exist.
    - 3NF: No transitive dependencies are present. Every non-key attribute depends directly on the primary key . And prime attribute also exists (RideID).
    - BCNF: Since RideID is a **primary key** and thus a **superkey** (it uniquely identifies each row).
- **Conclusion: The RideBooking table is in BCNF and 3NF.**

---

6. Payment Table:

- **FDs:**
    - PaymentID → Fare, RideID, PaymentTimestamp

      From these we can make various 3NF and BCNF FD's.
    - PaymentID -> RideID**(3NF) (1 FD)**
    - PaymentID -> Fare, PaymentTimestamp **(BCNF)(2 FD's are possible)**

- **Analysis:**
    - 1NF: All attributes (PaymentID, Fare, RideID, PaymentTimestamp) are atomic.
    - 2NF: PaymentID is the primary key, and all other attributes depend fully on it, so it's in 2NF.
    - 3NF: No transitive dependencies exist and prime attribute also exists.
    - BCNF: Since PaymentID is the **primary key** (and thus a superkey), this functional dependency satisfies the BCNF condition.
- **Conclusion: The Payment table is in BCNF and 3NF.**

---

7. Feedback Table:

- **FDs:**
    - FeedbackID → Rating, CustomerID, Comment

      From these we can make various 3NF and BCNF FD's.
    - FeedbackID -> CustomerID**(3NF) (1 FD)**
    - FeedbackID ->Rating,Comment **(BCNF)(2 FD's are possible)**

- **Analysis:**
    - 1NF: All attributes (FeedbackID, Rating, CustomerID, Comment) are atomic.

- 2NF: FeedbackID is the primary key, and all other attributes depend fully on it, so it's in 2NF.
- 3NF: No transitive dependencies exist and prime attribute also exists.
- BCNF: Since FeedbackID is the **primary key** (and therefore a **superkey**), this functional dependency satisfies the BCNF condition.

- **Conclusion: The Feedback table is in BCNF and 3NF.**

---

8. DriverTaxiCoordinator (DTC) Table:

- **FDs:**
  - DTCID → Name, PhoneNo, Email, LoginID

    From these we can make various 3NF and BCNF FD's.
  - DTCID -> LoginID **(3NF) (1 FD)**
  - DTCID -> Name, PhoneNo, Email **(BCNF)(3 FD's are possible)**

- **Analysis:**
  - 1NF: All attributes (DTCID, Name, PhoneNo, Email, LoginID) are atomic.
  - 2NF: DTCID is the primary key, and all other attributes depend fully on it, so it's in 2NF.
  - 3NF: No transitive dependencies exist**.**
  - BCNF: Since DTCID is the **primary key** (and therefore a superkey), this functional dependency satisfies the BCNF condition.
- **Conclusion: The DriverTaxiCoordinator table is in BCNF and 3NF.**

---

9. Taxi Table:

- **FDs:**
  - PlateNumber → Model, DTCID

    From these we can make various 3NF and BCNF FD's.
  - PlateNumber -> DTCID **(3NF) (1 FD)**
  - PlateNumber -> Model **(BCNF)(1 FD).**
- **Analysis:**
  - 1NF: All attributes (PlateNumber, Model, DTCID) are atomic.
  - 2NF: PlateNumber is the primary key, and all other attributes depend fully on it, so it's in 2NF.

- 3NF: No transitive dependencies exist. DTCID is a prime attribute.
- BCNF: Since Plate Number is primary key (and therefore a super key).

- **Conclusion: The Taxi table is in BCNF and 3NF.**

---

10. Gets Table (Composite Key: DriverID, DTCID, PlateNumber)

- **FDs:**
  - (DriverID, DTCID, PlateNumber) → No further dependencies

  As this FD is not meaningful so we will have to create new primary key for this table

  Our new table would be

Corrected Gets Table (Composite Key: GetsID,DriverID, DTCID, PlateNumber)

Functional Dependencies (FDs)

1. GetsID → DriverID (3NF)
2. GetsID → DTCID (3NF)
3. GetsID → PlateNumber (3NF)
4. (DriverID, DTCID, PlateNumber) → GetsID (3NF)

Analysis

- **1NF:**
  - All attributes are atomic; no repeating groups or arrays present.
  - The table is in 1NF, and there are no partial dependencies because GetsID is the primary key, and all non-key attributes depend on it.
- **2NF:**
  - The table is in 2NF, and there are no transitive dependencies; all non-key attributes are only dependent on the primary key (GetsID).
- **3NF:**
  - While the table is in 3NF, there are non-key functional dependencies (DriverID, DTCID, PlateNumber) which indicate that the primary key does not determine all attributes fully.
- **BCNF:**
  - **This means that the table does not satisfy BCNF because some non-trivial functional dependencies do not have a superkey as their determinant.**
- **Conclusion: The Gets table is in 3NF but not in BCNF due to non-key dependencies on DriverID, DTCID, and PlateNumber. To achieve BCNF, the table may need to be decomposed into multiple tables to eliminate these non-key dependencies**.

List of All tables with their corresponding normal forms:-

- Login: BCNF
- Driver: 3NF/BCNF
- Customer: 3NF/BCNF
- Shift: BCNF
- RideBooking: 3NF/BCNF
- Payment: 3NF/BCNF
- Feedback: 3NF/BCNF
- DriverTaxiCoordinator: 3NF/BCNF
- Taxi: 3NF/BCNF
- Gets: 3NF

We have changed the whole model is in 3NF/BCNF .

Anomalies and Redundancies in Taxi Management System Database

1 Anomalies in the Original Database Design

Based on the schema provided, here are the possible **update**, **delete**, and **insert anomalies**, along with scrutiny of redundancies.

1.1 Update Anomalies

a) **Foreign Key Dependencies**:
If a field that is part of a foreign key is updated, cascading changes or inconsistencies might occur across the tables.

- **Driver and Login Information**: The Driver and Customer tables reference the Login table through the LoginID. If login details such as LoginID need to be updated, all related entries in Driver, Customer, RideBooking, etc., must be updated. Failure to update any of them might cause inconsistencies.
- **Taxi and Coordinator Relationship**: Updating the DTCID in the DriverTaxiCoordinator table would require updating the Taxi and Gets tables to ensure no orphan records are left behind.

b) **Redundant Data Updates**:
When data is stored redundantly across multiple tables, updates can be cumbersome.

- **Driver's Phone Number and Email**: Both phone number and email are stored in the Driver, Customer, and DriverTaxiCoordinator tables. Updating contact details requires changing them in all relevant tables.

1.2 Delete Anomalies

a) **Cascade Deletion Issues**:
Cascade deletions can lead to the loss of critical data, especially when deletions affect records that are important for maintaining historical accuracy.

- **Cascade Deletion of Drivers or Customers**: In the RideBooking table, if a driver or customer is deleted (ON DELETE SET NULL), their associated records in RideBooking will still exist but with NULL values for DriverID and CustomerID. This might lead to incomplete or inconsistent data in ride history.

- **Feedback Loss on Customer Deletion**: When a customer is deleted, all their feedback is removed due to the ON DELETE CASCADE constraint in the Feedback table, leading to the loss of valuable feedback.

b) **Deletion of Coordinators Affects Taxi and Assignments**:
If a DriverTaxiCoordinator is deleted, cascading deletions in both the Taxi and Gets tables will result in the removal of taxi assignments and records (ON DELETE CASCADE). This deletion could unintentionally cause a loss of taxi records.

1.3 Insert Anomalies

a) **Dependency on Other Records**:
Insert anomalies arise when a new record cannot be added unless related records already exist.

- RideBooking Without Driver or Customer: A ride cannot be booked (RideBooking table) unless both a DriverID and CustomerID are already present. If you attempt to insert a ride without inserting the corresponding driver and customer first, the foreign key constraints will prevent the insertion.

b) **Assigning a Taxi Requires Pre-existing Records**:
The Gets table associates drivers with taxis and coordinators. For an entry to be inserted, related records (from the Driver, Taxi, and DriverTaxiCoordinator tables) must exist. If any of the foreign key dependencies is missing, the insert operation will fail.

2 Discussion of Redundancies

2.1 Redundant Data Across Tables

a) **Login Data Across Multiple Tables**:
The LoginID is used in multiple tables like Driver, Customer, and DriverTaxiCoordinator. While necessary for relational integrity, this creates redundancy where each entity is associated with similar fields like Username, PasswordHash, and Role.

b) **Contact Information**:
Both the Driver, Customer, and DriverTaxiCoordinator tables store fields like PhoneNo and Email. Since these fields are unique, any update in contact details must be propagated across all related tables, creating potential redundancy.

2.2 Foreign Key Redundancy

a) **Multiple Foreign Keys in RideBooking Table**:
The RideBooking table includes foreign keys to both the Driver and Customer tables. Although these relationships are necessary, having foreign keys across multiple tables can increase the complexity of maintaining relational integrity.

2.3 Redundant Cascade Deletion

a) **Cascading Deletes Across Multiple Relationships**:
The Gets table involves three foreign keys (DriverID, DTCID, and PlateNumber), each triggering a CASCADE on deletion. While this ensures consistency, it can lead to over-deletion, where the removal of one entity (e.g., a driver) could cause unintended cascading deletions in related tables.

3 Suggestions to Address Anomalies and Redundancies

1. **Avoid Cascade Deletes in Critical Relationships**:
Consider replacing ON DELETE CASCADE with ON DELETE RESTRICT or ON DELETE SET NULL in critical tables like RideBooking, where historical data is important. This will prevent unintentional loss of data when foreign key dependencies are deleted.

2. **Introduce Audit Logging for Updates and Deletes**:
Instead of cascading deletions, use soft deletes or audit logs to keep track of deleted records for historical purposes. For example, rather than deleting a driver or customer outright, mark them as inactive and archive their records.

3. **Normalize Data Further**:
Eliminate redundancies by further normalizing columns. For example, instead of storing PhoneNo and Email in each of the Driver, Customer, and DriverTaxiCoordinator tables, create a separate ContactInfo table and link it with foreign keys.

4. **Enforce Referential Integrity Without Redundancy**:
Reconsider whether relationships like between DriverTaxiCoordinator and Taxi need to trigger cascading deletions, or if there are better ways to handle changes without affecting the rest of the database.